

Week - 3 Divide and Conquer

Q1.

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     int n;
5     scanf("%d",&n);
6     int arr[n];
7     int count = -1;
8
9     for(int i = 0; i < n; i++)
10    {
11        scanf("%d",&arr[i]);
12    }
13
14    int left = 0;
15    int right = n-1;
16    int target = 0;
17    while(left <= right)
18    {
19        int mid = left + (right-left)/2;
20        if(arr[mid] == target)
21        {
22            count = mid;
23            right = mid - 1;
24        }
25        else
26        {
27            left = mid + 1;
28        }
29    }
30
31    if(count != -1)
32    {
33        printf("%d",n-count);
34    }
35    else
36    {
37        printf("0");
38    }
39
40
41
42 }
```

	Input	Expected	Got	
✓	5	2	2	✓
	1			
	1			
	1			
	0			
	0			
✓	10	0	0	✓
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
	1			
✓	8	8	8	✓

Q2.

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`

Output: 3

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`

Output: 2

Constraints:

- `n == nums.length`
- $1 \leq n \leq 5 * 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (pending review, v 70)

```
1 #include <stdio.h>
2 int main()
3 {
4     int n;
5     scanf("%d",&n);
6
7     int arr[n];
8     for(int i = 0; i < n; i++)
9         scanf("%d",&arr[i]);
10
11     int count = 0;
12     int candidate;
13
14     for(int i = 0; i < n; i++)
15     {
16         if(count == 0)
17             candidate = arr[i];
18         if(arr[i] != candidate)
19             count--;
20         else
21             count++;
22     }
23
24     printf("%d", candidate);
25
26
27
28 }
```

	Input	Expected	Got	
✓	3	3	3	✓
	3 2 3			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Q3.

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int findfloor(int arr[],int low,int high,int x)
4 {
5     if(low > high)
6         return -1;
7
8     if(x==arr[high])
9         return arr[high];
10
11     int mid = low + (high-low)/2;
12
13     if(arr[mid]==x)
14         return arr[mid];
15
16     if(x>arr[mid])
17     {
18         int rightfloor = findfloor(arr,mid+1,high,x);
19         if(rightfloor != -1)
20             return rightfloor;
21         return arr[mid];
22     }
23
24     return findfloor(arr,low,mid-1,x);
25 }
26
27 int main()
28 {
29     int n;
30     scanf("%d",&n);
31
32     int arr[n];
33     for(int i = 0;i<n;i++)
34     {
35         scanf("%d",&arr[i]);
36     }
37
38     int x;
39     scanf("%d",&x);
40
41     int floorvalue = findfloor(arr,0,n-1,x);
42     printf("%d",floorvalue);
43 }
```

	Input	Expected	Got	
✓	6	2	2	✓
	1			
	2			
	8			
	10			
	12			
	19			
	5			
✓	5	85	85	✓
	10			
	22			
	85			
	108			
	129			
	100			
✓	7	9	9	✓

Q4.

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int findTwoSum(int arr[], int left, int right, int x) {
5     if (left >= right) {
6         return 0;
7     }
8
9     int current_sum = arr[left] + arr[right];
10
11     if (current_sum == x) {
12         printf("%d\n%d\n", arr[left], arr[right]);
13         return 1;
14     }
15     else if (current_sum > x) {
16         return findTwoSum(arr, left, right - 1, x);
17     }
18     else {
19         return findTwoSum(arr, left + 1, right, x);
20     }
21 }
22
23 int main() {
24     int n;
25     if (scanf("%d", &n) != 1 || n <= 0) {
26         return 0;
27     }
28
29     int *arr = (int *)malloc(n * sizeof(int));
30     for (int i = 0; i < n; i++) {
31         scanf("%d", &arr[i]);
32     }
33
34     int x;
35     scanf("%d", &x);
36
37     if (!findTwoSum(arr, 0, n - 1, x)) {
38         printf("No\n");
39     }
40
41     free(arr);
42     return 0;
43 }
```

	Input	Expected	Got	
✓	4	4	4	✓
	2	10	10	
	4			
	8			
	10			
	14			
✓	5	No	No	✓
	2			
	4			
	6			
	8			
	10			
	100			

Passed all tests! ✓

Correct

Q5.

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5	12 34 67 78 98
67 34 12 98 78	

Answer:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void swap(int *a, int *b) {
5     int temp = *a;
6     *a = *b;
7     *b = temp;
8 }
9
10 int partition(int arr[], int low, int high) {
11     int pivot = arr[high];
12     int i = (low - 1);
13
14     for (int j = low; j < high; j++) {
15         if (arr[j] < pivot) {
16             i++;
17             swap(&arr[i], &arr[j]);
18         }
19     }
20     swap(&arr[i + 1], &arr[high]);
21     return (i + 1);
22 }
23
24 void quickSort(int arr[], int low, int high) {
25     if (low < high) {
26         int pi = partition(arr, low, high);
27
28         quickSort(arr, low, pi - 1);
29         quickSort(arr, pi + 1, high);
30     }
31 }
32
33 int main() {
34     int n;
35     if (scanf("%d", &n) != 1 || n <= 0) {
36         return 0;
37     }
38
39     int *arr = (int *)malloc(n * sizeof(int));
40     for (int i = 0; i < n; i++) {
41         scanf("%d", &arr[i]);
42     }
43
44     quickSort(arr, 0, n - 1);
45
46     for (int i = 0; i < n; i++) {
47         printf("%d ", arr[i]);
48     }
49     printf("\n");
50
51     free(arr);
52     return 0;
53 }
```

	Input	Expected	Got	
✓	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✓
✓	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✓
✓	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.